

Guia de Formação Técnica — Do Estudante ao Perfil Sênior neste Projeto

Objetivo: te dar um caminho completo para **entender profundamente** o projeto e evoluir sua maturidade técnica no contexto de microsserviços + microfrontends + operações.

Este guia foi construído com base no código atual do repositório, não em teoria genérica.

0) O que significa “ser sênior” aqui

Neste projeto, perfil sênior não é “saber mais sintaxe”. É conseguir:

1. **Explicar e justificar decisões** (trade-offs, não só implementação).
2. **Operar e depurar o sistema** de ponta a ponta sob falha.
3. **Evoluir arquitetura sem quebrar o que funciona**.
4. **Garantir qualidade** (testes, observabilidade, segurança, CI).
5. **Priorizar com pragmatismo** sob restrição de tempo.

Se você conseguir fazer isso, você já está atuando em nível sênior nesse contexto.

1) Mapa mental do sistema (visão de alto nível)

1.1 Serviços e responsabilidades

- **users-service** (FastAPI, porta 8001)
 - Cadastro/listagem de usuários.
 - Login e emissão de JWT.
- **orders-service** (FastAPI, porta 8002)
 - Lista/cria pedidos, consulta por ID, atualiza status.
 - Cache com Redis na listagem.
 - Integração opcional com IA para prioridade/resumo.
- **shell** (React host, porta 3000)
 - Fluxo de autenticação e rotas protegidas.
 - Carrega o MFE remoto de pedidos.
- **mfe-orders** (React remote, porta 3001)
 - UI de listagem/filtro/criação/detalhe de pedidos.

1.2 Dados e infraestrutura

- Banco PostgreSQL por serviço (isolamento de domínio):
 - **users-db**
 - **orders-db**
- Redis compartilhado como camada de cache no serviço de pedidos.
- Orquestração com **docker-compose.yml**.
- CI com GitHub Actions por contexto (users, orders, frontend).

2) Leitura guiada do código (ordem ideal)

2.1 Comece por infraestrutura

1. `docker-compose.yml`
 - Entenda dependências, portas, variáveis, health checks.
2. `services/*/Dockerfile` e `frontend/*/Dockerfile`
 - Entenda build/runtime de backend e frontend.

2.2 Backend users

1. `services/users/app/config.py`
2. `services/users/app/database.py`
3. `services/users/app/models.py`
4. `services/users/app/schemas.py`
5. `services/users/app/auth.py`
6. `services/users/app/routes/auth.py`
7. `services/users/app/routes/users.py`
8. `services/users/app/main.py`

2.3 Backend orders

1. `services/orders/app/config.py`
2. `services/orders/app/models.py`
3. `services/orders/app/schemas.py`
4. `services/orders/app/redis_client.py`
5. `services/orders/app/ai_service.py`
6. `services/orders/app/auth.py`
7. `services/orders/app/routes/orders.py`
8. `services/orders/app/main.py`

2.4 Frontend shell + MFE

1. `frontend/shell/webpack.config.js`
2. `frontend/mfe-orders/webpack.config.js`
3. `frontend/shell/src/context/AuthContext.jsx`
4. `frontend/shell/src/App.jsx`
5. `frontend/mfe-orders/src/api.js`
6. `frontend/mfe-orders/src/OrdersApp.jsx`
7. `frontend/mfe-orders/src/components/*.jsx`

2.5 Qualidade e entrega

1. `services/users/tests/*`
 2. `services/orders/tests/*`
 3. `.github/workflows/*.yaml`
-

3) Domínio técnico por camada (o que você precisa dominar)

3.1 FastAPI + SQLAlchemy + Pydantic

Você precisa dominar:

- Ciclo request → validação (`schemas`) → persistência (`models`) → resposta (`response_model`).
- Injeção de dependência (`Depends(get_db)`).
- Erros HTTP (`HTTPException`) e status codes corretos.
- Diferença entre schema de entrada e schema de saída.

No projeto:

- Entrada de criação em `OrderCreate/UserCreate`.
- Resposta serializada por `OrderResponse/UserResponse`.
- Tabelas criadas no startup via `Base.metadata.create_all(...)`.

Risco técnico que um sênior enxerga:

- `create_all` é prático para PMV, mas em produção madura o ideal é migração versionada (Alembic efetivamente usado em pipeline de schema).

3.2 Autenticação JWT

Você precisa dominar:

- Emissão (`users-service`) vs validação (`orders-service`).
- Claims úteis (`sub`, `email`, `role`, `exp`).
- Estratégias de autenticação distribuída.

No projeto:

- JWT emitido com `HS256` e segredo compartilhado.
- `orders-service` valida localmente (sem chamada inter-serviço).
- `PATCH /orders/{id}/status` exige token.

Trade-off real:

- Shared secret simplifica PMV e reduz latência.
- Em ambiente maior, pode evoluir para JWKS/OAuth2 para rotação/revogação melhor.

3.3 Cache e consistência

Você precisa dominar:

- Quando cachear leitura.
- Como invalidar no write.
- Staleness e consistência eventual.

No projeto:

- Cache de listagem em chave parametrizada por status/skip/limit.

- Invalidação por pattern no create/update status.
- Fallback resiliente se Redis indisponível.

Trade-off:

- `keys(pattern)` funciona para PMV, mas pode escalar mal em produção de alto volume.

3.4 IA aplicada a backend

Você precisa dominar:

- IA como componente opcional e não crítico.
- Timeouts/falhas e fallback determinístico.
- Validação/normalização de saída do modelo.

No projeto:

- IA usada para enriquecer pedido (prioridade + resumo), não bloquear criação.
- Sem API key ou com erro: fallback `medium/None`.
- Normalização de prioridade para evitar valores fora do domínio.

Sênior mindset:

- IA não pode derrubar operação central.

3.5 Microfrontends com Module Federation

Você precisa dominar:

- Host vs Remote.
- Compartilhamento de dependências singleton.
- Acoplamento entre shell e MFE (contratos de rota/módulo).

No projeto:

- Shell importa `mfe_orders/OrdersApp` dinamicamente.
- MFE expõe `./OrdersApp`.
- React e router compartilhados.

Risco técnico:

- Quebra de compatibilidade entre versões compartilhadas se não houver governança.

3.6 Frontend state + auth

Você precisa dominar:

- Estado de sessão no cliente.
- Persistência segura e UX de rota protegida.

No projeto:

- `AuthContext` guarda token/user em `localStorage`.

- `ProtectedRoute` bloqueia acesso sem token.

Trade-off:

- Simples e funcional; em produção avançada pode evoluir para estratégia de refresh token + proteção adicional.

3.7 Testes e estratégia

Você precisa dominar:

- Unit/integration boundary.
- O que mockar e por quê.
- O que ainda não está coberto.

No projeto:

- Users: valida login/cadastro/listagem/health.
- Orders: valida criação/lista/filtro/get/not found/auth no patch.
- Orders testa com mocks de cache e IA para previsibilidade.

Gap típico de maturidade (normal para PMV):

- Ausência de testes E2E ponta a ponta entre serviços com infraestrutura real.

3.8 CI e engenharia de entrega

Você precisa dominar:

- Pipelines segmentados por contexto.
- Feedback rápido por mudança.

No projeto:

- Workflow de users roda em mudanças de `services/users/**`.
- Workflow de orders roda em mudanças de `services/orders/**`.
- Frontend valida build em mudanças de `frontend/**`.

Sênior next step:

- Adicionar quality gates (lint, coverage threshold, security scan, dependency audit).

4) Plano de estudo progressivo (8 semanas)

Semana 1 — Compreensão funcional total

- Subir stack, rodar seed, testar fluxo na UI.
- Fazer mapa em papel: quem chama quem, onde token nasce e onde é validado.
- Entregar para si mesmo: explicar o projeto sem abrir código por 5 minutos.

Semana 2 — Backend users profundo

- Reimplementar mentalmente `POST /auth/login` linha a linha.
- Entender hashing e claims JWT.
- Exercício: desenhar 3 cenários de falha e resposta HTTP esperada.

Semana 3 — Backend orders profundo

- Seguir o fluxo de criação de pedido e cache.
- Entender impacto de `get_optional_user` vs `get_current_user`.
- Exercício: explicar por que o patch exige auth e o post não.

Semana 4 — Frontend e MFEs

- Entender rota protegida, lazy loading e módulo remoto.
- Exercício: explicar para alguém como o Shell encontra o remote em runtime.

Semana 5 — Testes e qualidade

- Rodar testes e estudar fixtures/mocks.
- Exercício: listar 5 testes que faltam e justificar prioridade.

Semana 6 — Operação e depuração

- Simular falhas (derrubar serviço, invalidar token, remover API key).
- Exercício: montar playbook de diagnóstico de incidente.

Semana 7 — Segurança e design review

- Revisar CORS, JWT secret, armazenamento de token, permissões.
- Exercício: apresentar 5 riscos + mitigação incremental.

Semana 8 — Narrativa sênior

- Treinar apresentação com foco em trade-offs, não só features.
- Exercício: responder 20 perguntas difíceis em voz alta.

5) Laboratórios práticos (aprender fazendo)

Lab 1 — Observabilidade mínima real

Objetivo: sair de logs apenas para visão operacional melhor.

- Adicionar request-id por requisição.
- Logar latência por endpoint.
- Expor métricas básicas (`/metrics`) no orders.

Aprendizado sênior:

- Telemetria não é “extra”, é requisito de operação.

Lab 2 — Segurança incremental

- Tornar `POST /orders/` autenticado (ou justificar publicamente por que não).
- Introduzir política de role para update de status.
- Revisar segredo JWT e estratégia de env.

Aprendizado sênior:

- Segurança é contexto + risco + custo.

Lab 3 — Teste de integração real

- Subir banco de teste em container.
- Executar suite apontando para PG real.
- Comparar achados vs SQLite.

Aprendizado sênior:

- Ambientes de teste muito “fáceis” escondem classes de bug.

Lab 4 — Evolução de contrato entre MFEs

- Definir checklist para mudanças de API consumida no frontend.
- Simular breaking change e mitigação (feature flag/versionamento).

Aprendizado sênior:

- Em arquitetura distribuída, governança de contrato é crítica.

6) Perguntas que um avaliador “sênior” pode fazer

1. Por que `POST /orders/` aceita usuário opcional?
2. Qual o impacto de cache stale na listagem?
3. Como você faria rotação de segredo JWT sem downtime?
4. O que acontece se a IA retornar JSON inválido?
5. Como garantir compatibilidade entre Shell e Remote em deploy independente?
6. Onde estão seus SLOs/SLIs?
7. Qual seu plano para incidentes em produção?
8. Como evitar regressão quando adicionar RBAC?
9. Como evoluir de shared secret para provedor de identidade?
10. Qual o custo operacional da arquitetura atual?

Treino ideal:

- Responder cada uma em 2 camadas:
 - curta (30s)
 - aprofundada (2 min)

7) Rubrica de autoavaliação (níveis)

Nível 1 — Júnior no projeto

- Consegue rodar localmente e explicar o fluxo principal.
- Faz mudanças pequenas sem quebrar tudo.

Nível 2 — Pleno no projeto

- Entende trade-offs das principais decisões.
- Consegue depurar falhas comuns sozinho.
- Escreve testes para novas features.

Nível 3 — Sênior no projeto

- Decide roadmap técnico por impacto/risco.
- Melhora arquitetura sem overengineering.
- Define padrões para equipe (contratos, qualidade, operação).
- Consegue defender decisões sob questionamento crítico.

8) Roteiro de estudo diário (90 minutos)

Bloco A (30 min) — Código

- Ler um arquivo-chave e anotar decisões implícitas.

Bloco B (30 min) — Prática

- Rodar cenário real (API, UI, falha, teste).

Bloco C (30 min) — Explicação

- Gravar áudio de 3–5 min explicando o que aprendeu como se fosse entrevista técnica.

Regra de evolução:

- Se não consegue explicar com clareza, ainda não dominou.

9) Checklist de domínio para sua apresentação

Você está pronto quando consegue:

- Explicar por que existem dois serviços e dois bancos.
- Explicar JWT end-to-end sem consultar código.
- Explicar cache e invalidação com exemplo concreto.
- Explicar integração de IA e fallback em falha.
- Explicar Module Federation (host/remote/shared).
- Explicar o que foi priorizado e o que ficou para depois com critério.
- Responder perguntas difíceis sem entrar em contradição com o código.

10) Próximo passo recomendado

Use este fluxo:

1. Leia [GUIA_APRESENTACAO_APOIO.md](#) para visão geral e narrativa.
2. Estude este guia ([GUIA_ESTUDO_SENIOR_PROJETO.md](#)) para aprofundamento.
3. Faça um ensaio técnico completo (20–30 min) com cronômetro.
4. Faça uma rodada de perguntas e respostas simulada.

Se quiser, eu posso gerar na sequência um **simulado de banca com 30 perguntas (fácil, média, difícil) e respostas modelo** em outro arquivo para treino final.